Travail Pratique 1

Contexte

Le but de ce travail est de créer un programme permettant de gérer des dossiers médicaux. Il permet de créer et afficher une liste de médecins et une liste de patients, ainsi que de planifier des rendez-vous.

Déroulement de l'exécution

Le programme **TP1.exe** distribué avec cet énoncé sert de référence comme résultat à atteindre. Assurez-vous que votre travail s'exécute de façon identique. Portez attention aux affichages, votre programme doit afficher les mêmes informations.

Au démarrage, votre programme affiche le menu principal.

```
==========================================================================
= Centre médical                                                         =
==========================================================================

 P) Affichage
 A) Ajout
 R) Prendre rendez-vous
 Q) Quitter
```

Option P : Affichage

Cette option permet d'afficher toute l'information contenue dans le système.

```
==========================================================================
= Centre médical - Affichage                                             =
==========================================================================

 P) Afficher les patients
 M) Afficher les médecins
 R) Retour au menu principal
```

Afficher les patients

Cette option affiche tous les patients du système, dans 2 catégories : ceux associés à un médecin, et ceux en attente d'un médecin.

Afficher les médecins

Un médecin ne peut prendre en charge qu'un maximum de 3 patients.

Cette option affiche tous les médecins du système, dans 2 catégories : ceux encore disponibles à prendre en charge des patients (car ils n'ont pas atteint la limite de 3), et ceux qui ont déjà atteint la limite de patients.

Option A : Ajout

Cette option permet d'ajouter des médecins et des patients.

```
==========================================================================
= Centre médical - Ajout                                                 =
==========================================================================

 P) Ajouter un patient
 M) Ajouter un médecin
 R) Retour au menu principal
```

Ajout d'un patient

Lors de l'ajout d'un patient, le programme demande le **Prénom** et le **Nom** du patient. Ce sont des chaines de caractères qui ne doivent pas être vides ni ne contenir que des espaces. Le programme redemande les informations tant qu'elles ne sont pas valides.

Le programme génère alors un identifiant pour le patient (un nombre entier) qui est unique pour toutes les personnes (patients et médecins).

Si un médecin est disponible pour prendre en charge un patient, le nouveau patient est assigné à ce médecin.

Sinon, le patient est placé en attente d'un médecin. L'ordre d'ajout doit être conservé, car l'assignation des patients doit être faite dans le même ordre lorsqu'un médecin deviendra disponible.

Ajout d'un médecin

Comme pour un patient, le programme demande le **Prénom** et le **Nom** du médecin, redemande les informations tant qu'elles ne sont pas valides, et génère un identifiant unique.

Si un patient est en attente d'un médecin, il est assigné au nouveau médecin. Si plus d'un patient est en attente, ils sont assignés au nouveau médecin, dans l'ordre dans lequel ils ont été ajoutés au système, jusqu'au maximum de **3**.

Si le médecin n'a pas atteint la limite de **3** patients, il est placé en disponibilité. Sinon, il n'est plus disponible pour prendre en charge de nouveaux patients.

Option R : Prise de rendez-vous

Cette option permet à un patient de prendre un rendez-vous avec un médecin.

Le programme demande premièrement l'identifiant du patient, suivi de la date et heure du rendez-vous, ainsi que la durée du rendez-vous (parmi 3 options).

Si le patient est pris en charge par un médecin, le rendez-vous est pris avec ce médecin. Sinon, le médecin ayant le plus petit nombre de rendez-vous est choisi. (En cas d'égalité, l'un des médecins ayant le plus petit nombre de rendez-vous est choisi.)

Le rendez-vous ne doit pas être en conflit avec un rendez-vous existant, ni du patient ni du médecin.

Le rendez-vous est ajouté à l'horaire du patient et du médecin, qui doit rester en ordre croissant. L'ajout du nouveau rendez-vous doit donc être fait à la bonne position.

Si une valeur donnée est invalide (mauvais format, patient inconnu, etc.) ou qu'il y a un conflit d'horaire, le rendez-vous est annulé, un message d'erreur est affiché et le programme retourne au menu.

Fichiers de données

Sauvegarde de l'information

Lorsque le programme se termine normalement (option « **Q** » du menu principal), les patients et les médecins sont sauvegardés dans **2** fichiers texte, l'un pour les médecins et l'un pour les patients. Seuls l'identifiant, le prénom et le nom des personnes sont sauvegardés.

Les rendez-vous ne sont pas sauvegardés.

Les données doivent être écrites dans les fichiers dans le même ordre qu'elles ont été entrées dans le système.

Chargement de l'information

Lors du lancement du programme, si l'un des fichiers, ou les deux fichiers sont présents, ils sont lus et l'information est chargée en mémoire.

IMPORTANT : Les fichiers ne sont lus qu'**une seule fois**, au lancement du programme, et ne sont écrits qu'une seule fois, lors de l'appel de la méthode `Save`, avant de terminer le programme.

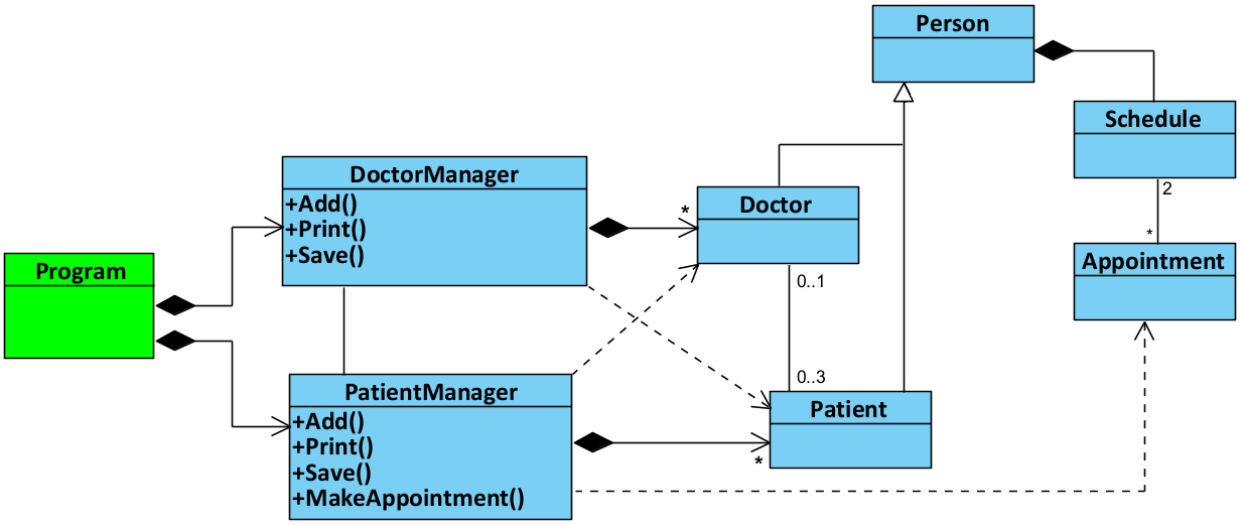
Si l'un des fichiers, ou les deux fichiers ne sont pas présents, alors le programme démarre sans information.

Exigences

- Le programme respecte les principes de la programmation orientée objet :
  - Encapsulation (données regroupées dans un objet)
    Il ne doit avoir **aucun** attribut public, **même** s'ils sont `readonly`.
  - Abstraction (présente une interface en cachant les détails d'implémentation)
    Les propriétés ne doivent être publiquement modifiables que si nécessaire.
  - Héritage (regroupement des fonctionnalités communes).
- Les attributs non modifiables après l'initialisation doivent être qualifiés de `readonly`.
- Utilisez des constantes, lorsqu'applicables.
- N'utilisez le niveau d'accès `protected` que lorsque ça fait du sens.
- Le programme doit valider toutes les entrées de l'utilisateur et résister à toute donnée erronée.
- Le programme ne doit pas planter, il doit gérer correctement toutes les erreurs. Si une situation fait en sorte que le programme ne peut continuer, il doit se terminer proprement en indiquant un message approprié à l'utilisateur.
- Vous ne devez utiliser que les notions vues en classe au moment de la distribution du TP (10 avril 2026). Les notions vues par la suite ne font pas partie de ce travail.
- Le fichier **Program.cs** fourni avec cet énoncé contient la méthode `Main` ainsi que les méthodes qui gèrent les menus et l'exécution générale du programme. Vous devez l'inclure dans votre projet et vous ne devez pas le modifier.

Classes requises

Le programme doit définir **au minimum** les classes suivantes (vous pouvez en définir d'autres si vous voulez) :



- La classe `Doctor` représente un médecin. Un objet de cette classe est créé à chaque ajout d'un médecin.
  - Doit avoir un constructeur qui demande l'information à l'utilisateur.
  - Doit avoir un constructeur qui extrait l'information lue du fichier de sauvegarde des médecins.
  - On doit pouvoir assigner jusqu'à `3` patients à un médecin.
- La classe `Patient` représente un patient. Un objet de cette classe est créé à chaque ajout d'un patient.
  - Doit avoir un constructeur qui demande l'information à l'utilisateur.
  - Doit avoir un constructeur qui extrait l'information lue du fichier de sauvegarde des patients.
  - On doit pouvoir assigner un médecin à un patient.
- La classe `Person` contient les attributs, propriétés et méthodes communes aux médecins et aux patients. Elle est responsable d'attribuer un identifiant unique à toutes les personnes.
- La classe `Appointment` contient l'information d'un rendez-vous entre un patient et un médecin.
- La classe `Schedule` contient tous les rendez-vous de la personne associée.
- La classe `DoctorManager` contient tous les médecins du système et gère les fonctionnalités reliées aux médecins : ajouter un médecin, afficher les médecins, lire et enregistrer le fichier de sauvegarde des médecins, etc. Un seul objet de cette classe existe dans le programme.
  - Le constructeur lit (s'il existe) le fichier de sauvegarde des médecins.
  - Pour que le fichier `Program.cs` compile, la classe doit implémenter les méthodes indiquées dans le diagramme.
- La classe `PatientManager` contient tous les patients du système. L'équivalent de la classe `DoctorManager`, mais pour gérer les patients.
  - Le constructeur reçoit comme paramètre une référence sur le gestionnaire de médecins.

Consignes

- Sur le site *GitHub*, créez un nouveau dépôt **privé** nommé « **TP1Prog00** ».
  - Dans l'onglet « **Settings** », dans la catégorie « **Collaborators** », cliquez le bouton vert « **Add people** ».
  - Dans la fenêtre qui s'ouvre, entrez « **jbeaudet-cstj** », puis cliquez le bouton « **Add jbeaudet-cstj to this repository** ».
- Clonez ce nouveau dépôt sur votre ordinateur.
- Dans *Visual Studio*, créez un nouveau projet nommé « **TP1** », dans une nouvelle solution.
  - Choisissez votre dépôt « **TP1Prog00** » comme emplacement.
  - Effectuez le travail dans ce projet.
- Assurez-vous d'avoir une solution qui compile même si elle n'est pas 100 % fonctionnelle, plutôt qu'une solution avec des erreurs.

Pondération

Ce travail compte pour **15** % de la note finale

Détermination correcte des attributs et des méthodes des classes	20 %
Application judicieuse des principes d'encapsulation et d'héritage	20 %
Implémentation correcte des classes dans le programme	20 %
Notation claire et pertinente de commentaires en nombre approprié dans le code informatique	10 %
Respect systématique des normes de programmation	5 %
Fonctionnement correct du programme	25 %

Pénalités

Français : Jusqu'à 10 % de pénalité

Retard : Pénalité de 10 % par jour de retard (incluant les fins de semaine)

Plagiat : Ceci est un travail individuel. Application de la Politique institutionnelle pour *toutes les personnes* impliquées.

REMISE

Retrouvez (dans l'historique de *GitHub* ou à l'aide de la commande « **`git log`** ») l'identifiant du commit correspondant à votre remise. Copiez-le.

Ouvrez l'application « **`Bloc-Notes`** » (*notepad*) et collez l'identifiant copié. Sauvegardez le fichier.

Vous devez remettre le fichier contenant l'identifiant dans LÉA, dans la rubrique Travaux, pour le TP .

Ne remettez rien d'autre dans LÉA. La correction sera effectuée dans le dépôt partagé. Tout code remis dans LÉA sera **ignoré et non corrigé**.

Vous devez remettre votre travail avant **`15:20`** le **`vendredi 24 avril 2026`**. La date de remise dans LÉA, et non la date du commit, sera prise en compte pour déterminer les pénalités de retard.